

The transcript is explaining **GitHub Actions dependency caching**—one of the most important optimizations for CI/CD pipelines. Here's a structured explanation.

Why do we need caching?

Every GitHub Actions **job runs on a brand-new virtual machine**.

Without caching:

Workflow Run #1



Runner A

npm ci

↓

Downloads all packages from npm registry

↓

Job ends

↓

Runner deleted

Next workflow:

Workflow Run #2



Runner B

npm ci

↓

Downloads everything again

↓

Slow

Every run downloads packages again because the previous runner no longer exists.

What does npm already do?

Even locally, npm already creates its own cache.

Example:

C:\Users\User\AppData\Local\npm-cache (Windows)



or

~/ .npm

(Linux/macOS)

When you install packages:

npm ci



npm stores downloaded packages in

~/ .npm



Next installation:

npm ci



Instead of downloading everything again, npm first checks its cache.

Unfortunately...

GitHub Actions runners are temporary.

So this cache disappears after every job.

GitHub Cache Action solves this

GitHub provides

</> YAML



actions/cache@v4

It saves folders into GitHub's cloud storage.

Instead of

Runner

|

| ~/ .npm

|

Deleted



It becomes

Runner

|

| ~/.npm

|

actions/cache

|

v

GitHub Cache Storage



Next job:

GitHub Cache Storage

|

v

Restore ~/.npm

|

Runner

|

npm ci



Now npm finds its cache immediately.

Where should cache step be placed?

Always before installation.

ChatGPT   Upgrade



Share



steps:

- uses: actions/checkout@v4

- name: Cache dependencies

 - uses: actions/cache@v4

 - with:

 - path: ~/.npm

 - key: npm-\${{ hashFiles('**/package-lock.json') }}

- name: Install dependencies

 - run: npm ci

Why?

Because:

```
Cache Restore
  ↓
npm ci
  ↓
Use cached packages
```



If cache doesn't exist:

```
Cache Restore
  ↓
Nothing found
  ↓
npm downloads packages
  ↓
Cache action saves them afterwards
```



Understanding path

```
</> YAML

path: ~/.npm
```



means

| Save the npm cache folder.

It **does not** save `node_modules`.

It saves:

```
~/.npm
```



Example contents:

```
~/.npm
├─ _cacache
├─ index-v5
├─ tmp
└─ logs
```



These are downloaded package archives.

Understanding the cache key

The important line:

</> YAML



```
key: npm-`${ hashFiles('**/package-lock.json') }`
```

Let's break it down.

Static key

If you wrote

</> YAML



```
key: npm-cache
```

Every workflow would reuse the same cache forever.

Problem:

package-lock.json changes



↓

GitHub still restores old cache

↓

Wrong dependencies

Bad idea.

Dynamic key

Instead:

</> YAML



```
hashFiles('**/package-lock.json')
```

GitHub calculates a hash.

Example:

Initial file:

```
package-lock.json
```



```
Angular 19
```

```
RxJS 8
```

```
Express 5
```

Hash:

```
6fd29a91
```



Key becomes

```
npm-6fd29a91
```



Later you update Angular.

```
Angular 20
```

```
RxJS 8
```

```
Express 5
```



Hash becomes

```
91af43d2
```



Now the key becomes

```
npm-91af43d2
```



GitHub sees:

```
Old cache:
```

```
npm-6fd29a91
```



```
New request:
```

```
npm-91af43d2
```

Not the same.

Therefore:

Old cache ignored



↓

npm downloads new packages

↓

New cache saved

Automatic cache invalidation.

Why `**/package-lock.json`?

The pattern

`**`



means

| Search recursively.

It matches

`package-lock.json`



or

`frontend/package-lock.json`



or

`apps/web/package-lock.json`



or

`packages/admin/package-lock.json`



Useful in monorepos.

What happens during the first run?

Cache Step



↓

Looks for cache

↓

Not Found

↓

npm ci

↓

Downloads everything

↓

Job finishes

↓

Post Cache Step

↓

Uploads ~/.npm to GitHub

Output:

Cache not found



...

Post Cache

Saving cache...

Second workflow run

Now:

Cache Step



↓

Cache Found

↓

Downloads ~/.npm

↓

npm ci

↓

Uses local cache

↓

Very fast

Instead of downloading from npm registry:

Internet



↓

download

↓

install

It becomes

GitHub Cache



↓

restore

↓

install

Much faster.

Why does `npm ci` still execute?

Many beginners think

If cache exists, `npm ci` will be skipped.

No.

It still runs.

The difference:

Without cache

```
npm ci
```

↓

Download 800 packages

↓

Install



With cache

```
npm ci
```

↓

Read packages from `~/.npm`

↓

Install



Installation still happens.

Downloading is avoided.

What is the "Post Cache" step?

You'll notice GitHub automatically creates another step.



You didn't write it.

The cache action does.

It executes after the job completes.

Flow:

Cache Restore



↓

Run your steps

↓

Job finished

↓

Post Cache

↓

Save updated cache

This ensures newly downloaded packages are uploaded for future runs.

Why repeat the cache step in every job?

Suppose you have:

Test Job



Build Job

Deploy Job

Each job gets a fresh machine.

Runner A



↓

Test

Runner A is destroyed.

Build runs on

Runner B



Runner B has nothing.

So Build must also restore the cache.

Therefore:

</> YAML



Test

Cache Restore

npm ci

Build

Cache Restore

npm ci

Deploy

Cache Restore

Each job restores the same shared GitHub cache independently.

Complete workflow example

</> YAML



```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - name: Cache npm
```

```
        uses: actions/cache@v4
```

```
with:
  path: ~/.npm
  key: npm-${{ hashFiles('**/package-lock.json') }}

- run: npm ci
- run: npm test

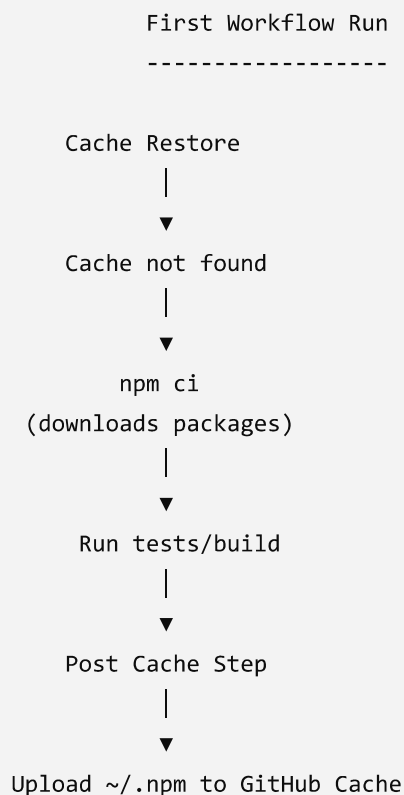
build:
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4

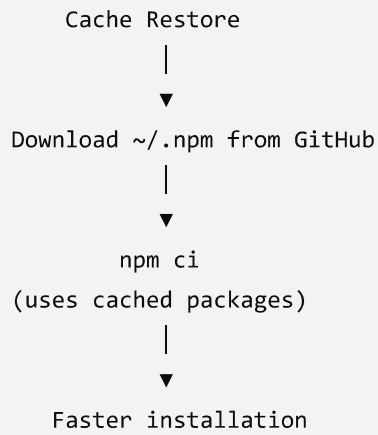
    - name: Cache npm
      uses: actions/cache@v4
      with:
        path: ~/.npm
        key: npm-${{ hashFiles('**/package-lock.json') }}

    - run: npm ci
    - run: npm run build
```

End-to-end flow



Second Workflow Run



Key takeaway: `actions/cache` does **not** cache your installed `node_modules` by default. It caches npm's **download cache** (`~/.npm`), allowing `npm ci` to reuse already-downloaded packages. The `hashFiles('**/package-lock.json')` function automatically invalidates the cache whenever your project dependencies change, ensuring both speed and correctness.

